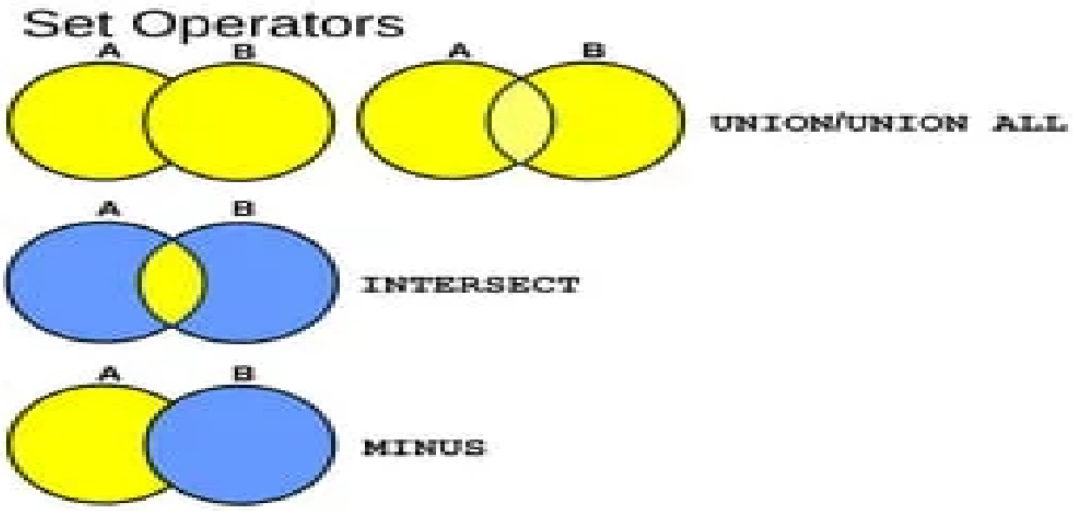
SET Operators

Union : Combines results from two queries and removes duplicate rows. Columns must have the same number and type.

Union All : Combines results from two queries and includes all rows, including duplicates. Columns must have the same number and type

Intersect : Returns rows that are common to both queries. Columns must have the same number and type.

EXCEPT: Returns rows from the first query that are not present in the second query. Columns must have the same number and type.




```
CREATE TABLE Students (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50),
   course VARCHAR(50)
);

INSERT INTO Students VALUES
```

```
(101,'Rahul','Java Full Stack'),
(102,'Priya','Python Full Stack'),
(103,'Kiran','Data Science'),
(104,'Sneha','Java Full Stack'),
(105,'Ajay','Testing');

CREATE TABLE Placed_Students (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50),
   company_name VARCHAR(50)
);
INSERT INTO Placed_Students VALUES
(103,'Kiran','TCS'),
(104,'Sneha','Infosys'),
(105,'Ajay','Wipro'),
(106,'Pooja','Accenture'),
(107,'Ramesh','Capgemini');


-- UNION Examples
SELECT STUDENT_NAME FROM STUDENTS
   UNION
   SELECT STUDENT_NAME FROM PLACED_STUDENTS;


-- UNION ALL Examples
SELECT STUDENT_NAME FROM STUDENTS
   UNION ALL
```

```
   SELECT STUDENT_NAME FROM PLACED_STUDENTS;
-- INTERSECT Examples
SELECT STUDENT_NAME FROM STUDENTS
   INTERSECT
   SELECT STUDENT_NAME FROM PLACED_STUDENTS;
-- MINUS Examples
SELECT STUDENT_NAME FROM STUDENTS
  EXCEPT
   SELECT STUDENT_NAME FROM PLACED_STUDENTS;
```

What is a Join?

A join is an SQL operation that combines rows from two or more tables into a single result set, based on a related column between them. In a relational database, data is split across several tables and connected by keys: a primary key in one table and a foreign key in another. A join follows those keys to bring the related data back together when we need it.

In a hospital system, patient details live in the patients table, doctor details in the doctors table, and each appointment stores only patient_id and doctor_id. To print a report such as "Rahul Verma visited Dr. Ravi Kumar on 10 June", we must join the three tables.

1.2 Why Do We Need Joins?

- Avoid data redundancy: a doctor's name and salary are stored once in doctors, not repeated in every appointment row (this is normalization).
- Combine related data: reports usually need columns from several tables at once, such as patient name, doctor name and fee.
- Maintain data integrity: updating a doctor's specialization in one place automatically reflects everywhere it is joined.
- Answer business questions: appointments per department, patients with no visits, doctors with no appointments, and so on.
- Save storage and improve consistency compared with keeping one giant table with repeated values.
- Flexibility: the same tables can be combined in many different ways depending on the question asked.

1.3 Advantages of Joins

- Retrieve related data from multiple tables in one query instead of several separate queries.
- Reduce duplicate data and storage because tables can stay normalized.
- Work directly in the database engine, which is optimized for joins and can use indexes.
- Support many types of relationships (one-to-one, one-to-many, many-to-many, hierarchical using self join).
- Outer joins reveal missing data such as unmatched or orphan records.
- Enable aggregation across tables (COUNT, SUM, AVG with GROUP BY).
- Standard SQL feature, portable across MySQL, PostgreSQL, SQL Server, Oracle and SQLite.

1.4 Disadvantages of Joins

- Performance cost: joining large tables without indexes on the join columns can be slow.
- Complexity: queries with many joins are harder to read, write and debug.
- Accidental Cartesian products: forgetting the join condition multiplies rows (m x n).
- NULL surprises: NULL values never match in equality joins, so rows can silently disappear in an INNER JOIN.
- Incorrect join type or duplicate keys can produce missing or duplicated rows and wrong totals.
- Higher memory and CPU usage for joins across very large tables; very heavy joins may need denormalization or caching.
- NATURAL JOIN can break silently when new columns with the same name are added.

types of joins in MySQL

1.inner join

2.left join

3. right join

4. full join

5. cross join

6. self join

1. INNER JOIN

Returns only the rows that have matching values in both tables.

Syntax

—----------

SELECT columns

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

2. LEFT JOIN (or LEFT OUTER JOIN)

Returns all rows from the left table and the matched rows from the right table. If no match, NULL values appear for right table columns.

Syntax

—------

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.common_column = table2.common_column;

3.RIGHT JOIN (or RIGHT OUTER JOIN)

Returns all rows from the right table and the matched rows from the left table. If no match, NULL values appear for left table columns.

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.common_column = table2.common_column;

4. FULL OUTER JOIN (Not directly supported in MySQL)

To simulate, use the UNION of LEFT JOIN and RIGHT JOIN.

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.common_column = table2.common_column

UNION

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.common_column = table2.common_column;

5. CROSS JOIN

Returns Cartesian product — every row from the first table with every row from the second table.

SELECT columns

FROM table1

CROSS JOIN table2;

6. SELF JOIN

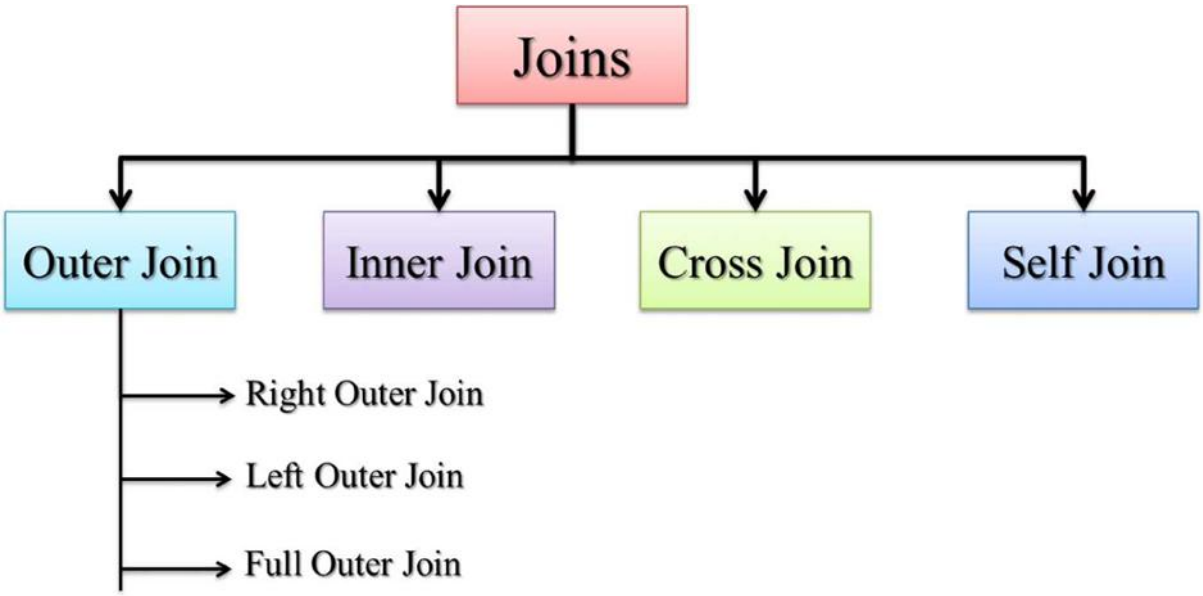
A table joined with itself.

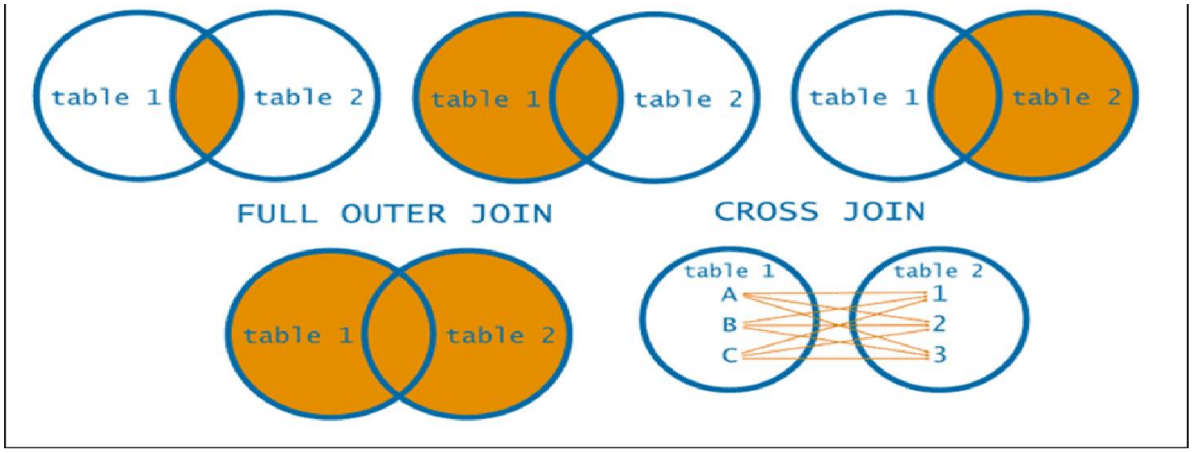
SELECT a.columns, b.columns

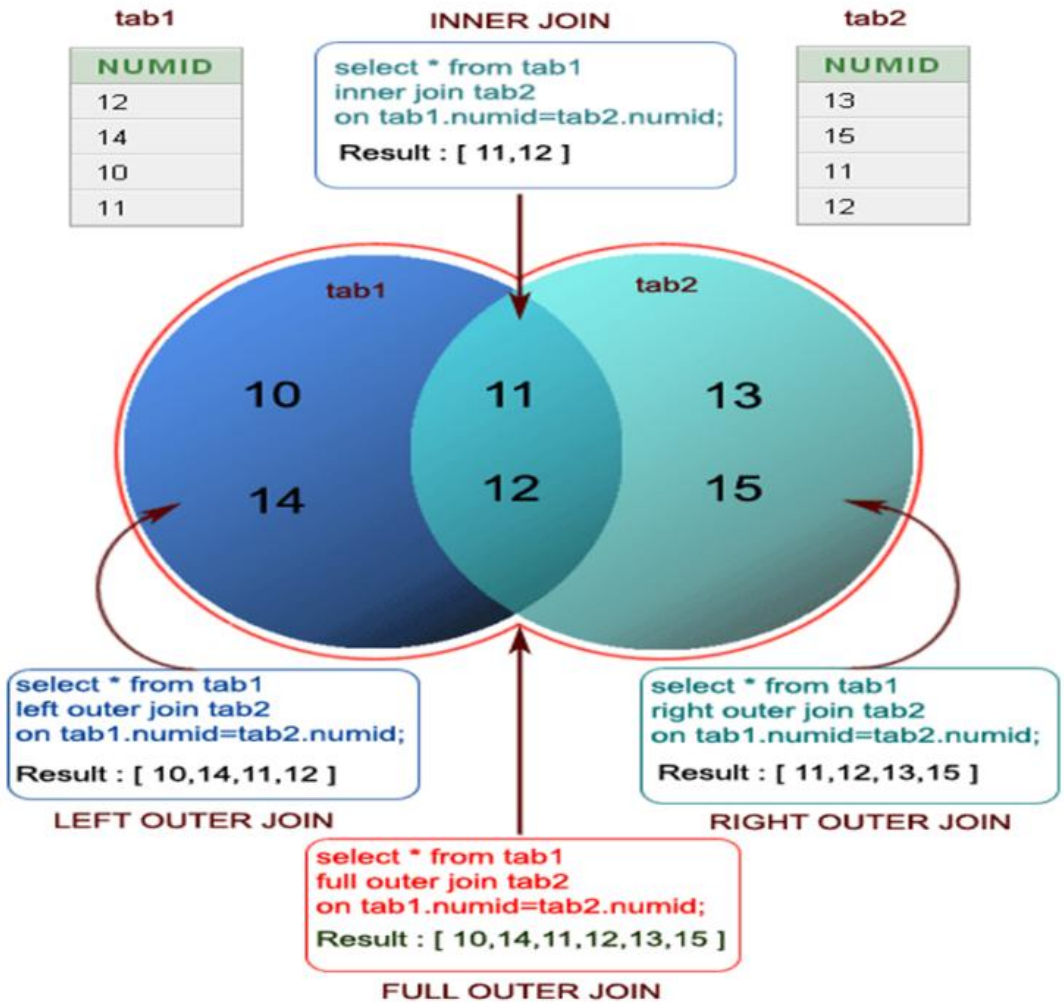
FROM table1 a

INNER JOIN table1 b

ON a.common_column = b.common_column;


Joins
Outer Join
Inner Join
Cross Join
Self Join
Right Outer Join
Left Outer Join
Full Outer Join


table 1
table 2
table 1
table 2
table 1
table 2
FULL OUTER JOIN
CROSS JOIN
table 1
table 2
table 1
table 2
A
B
C
1
2
3



CREATE TABLE TAB1(NUMID INT(3));

CREATE TABLE TAB2 (NUMID INT(3));

INSERT INTO TAB1 VALUES(10);

INSERT INTO TAB1 VALUES(11);

INSERT INTO TAB1 VALUES(12);

INSERT INTO TAB1 VALUES(14);

INSERT INTO TAB2 VALUES(11);

INSERT INTO TAB2 VALUES(12);

INSERT INTO TAB2 VALUES(13);

INSERT INTO TAB2 VALUES(15);

SELECT * FROM TAB1;

SELECT* FROM TAB2;

INNER JOIN:- • Only matched records in both the tables

SELECT*FROM TAB1 INNER JOIN TAB2

ON

TAB1.NUMID=TAB2.NUMID

LEFT JOIN:- Matched records+ unmatched from left table dept

SELECT*FROM TAB1 LEFT JOIN TAB2

ON

TAB1.NUMID=TAB2.NUMID

RIGHT JOIN:- Matched records+ unmatched from right table emp

SELECT*FROM TAB1 RIGHT JOIN TAB2

ON

TAB1.NUMID=TAB2.NUMID

FULL JOIN:- Matched records + unmatched records from both the tables

FULL JOIN NOT SUPPORTED mysql:

CrossJoin:- SELECT *

FROM TAB1

CROSS JOIN TAB2;